CSC3002 - Strings: Detailed Summary

> Based on **4-CSC3002 Lecture Notes - Strings** (39 slides).
> This first section is a quick-reference page. Detailed explanations and examples follow.

Page 1 - Important Functions and Operators

C++ `std::string` (`#include <string>`)

Operation	Purpose	Example / result
`str.length()`	Number of characters	`"hello".length()` -> `5`
`str[index]`	Read or modify a character; no range check	`str[0] = 'H';`
`str.at(index)`	Read or modify a character with range checking	`str.at(0)`
`str.substr(pos, len)`	Return `len` characters starting at `pos`	`"hello".substr(1, 3)` -> `"ell"`
`str.find(ch, pos)`	Find a character at or after `pos`	Returns an index or `string::npos`
`str.find(text, pos)`	Find a substring at or after `pos`	`str.find("cat", 0)`
`str.erase(pos, count)`	Delete `count` characters	`str.erase(2, 3)`
`str.insert(pos, text)`	Insert text at position `pos`	`str.insert(1, "abc")`
`str.replace(pos, count, text)`	Replace part of a string	`str.replace(0, 2, "Hi")`
`str.c_str()`	Obtain a null-terminated C-style view	`strlen(str.c_str())`
`s1 + s2`	Return concatenated string	`string("Hi") + "!"`
`s1 += s2`	Append to `s1`	`s1 += "!";`
`s1 = s2`	Copy `s2` into `s1`	`name = "Ray";`
`==`, `!=`, `<`, `<=`, `>`, `>=`	Lexicographic comparison	`"ant" < "bat"` is true

Character functions (`#include <cctype>`)

Function	Meaning
`isdigit(ch)`	Is `ch` a digit?
`isalpha(ch)`	Is `ch` a letter?
`isalnum(ch)`	Is `ch` a letter or digit?
`islower(ch)` / `isupper(ch)`	Is `ch` lowercase / uppercase?
`isspace(ch)`	Is `ch` whitespace?
`tolower(ch)` / `toupper(ch)`	Return lowercase / uppercase equivalent if available

Essential C-string functions (`#include <cstring>`)

Function	Purpose	Important condition
`strlen(s)`	Length before the first `\0`	`s` must be null-terminated
`strcpy(dest, src)`	Copy `src` into `dest`	`dest` must have enough space
`strncpy(dest, src, n)`	Copy at most `n` characters	May not add `\0` if truncated
`strcat(dest, src)`	Append `src` to `dest`	`dest` needs enough total space
`strncat(dest, src, n)`	Append at most `n` characters	`dest` still needs enough space

Function	Purpose	Important condition
`strcmp(a, b)`	Lexicographically compare strings	Returns `< 0`, `0`, or `> 0`
`strncmp(a, b, n)`	Compare up to `n` characters	Same return convention
`strchr(s, ch)` / `strrchr(s, ch)`	Find first / last occurrence of `ch`	Returns pointer or null pointer
`strstr(s, text)`	Find the first substring	Returns pointer or null pointer
`strtok(s, delimiters)`	Split a modifiable C string into tokens	Destructive; maintains internal state

> **Default choice:** use `std::string` for ordinary C++ programs. Use C strings mainly when required by C libraries or older interfaces.

---

1. Strings in C and C++

Text processing appears in nearly every program, from displaying instructions to processing language data. Conceptually, a string is a sequence of characters, but C and C++ provide different abstractions:

- **C:** a string is an array of `char` ending with a null character, `\0`.
- **C++:** `std::string` is a class that hides storage details and provides operators and methods.

This contrast reflects two programming styles. C exposes the representation and operates on it through separate functions; C++ packages data and behavior together in an object.

The C++ Standard Library places its features in the `std` namespace and includes containers, algorithms, strings, streams, localization, threads, numerics, and adapted C libraries such as `<cmath>`, `<cctype>`, and `<cstring>`.

---

2. Characters and Encoding

2.1 The `char` type

A `char` occupies one byte. The lecture uses ASCII to explain character codes. Standard ASCII uses values `0` - `127`; an eight-bit byte has 256 bit patterns, and values above `127` depend on the extended encoding in use.

```
char ch1 = 'a';  // character literal
char ch2 = 97;   // ASCII code for 'a'
char ch3 = 32;   // ASCII code for a space
char ch4 = ' ';  // equivalent to the previous line in ASCII
```

Useful ASCII ranges are:

- Digits: `'0'` - `'9'` -> decimal `48` - `57`
- Uppercase letters: `'A'` - `'Z'` -> decimal `65` - `90`
- Lowercase letters: `'a'` - `'z'` -> decimal `97` - `122`

Unicode supports far more writing systems than ASCII. The lecture mentions `wchar_t` as a wider character type but keeps its examples focused on `char`.

2.2 Control characters and escape sequences

Decimal	Escape	Meaning
`0`	`\0`	Null terminator
`9`	`\t`	Horizontal tab
`10`	`\n`	Line feed / newline
`13`	`\r`	Carriage return

Historically, line endings differ by platform: Unix-like systems use `\n`, Windows uses `\r\n`, and classic Mac systems used `\r`.

2.3 `<cctype>` classification and conversion

The `<cctype>` functions let a program describe its intent clearly instead of manually comparing ASCII ranges.

```
#include <cctype>

if (std::isalpha(ch)) { /* it is a letter */ }
if (std::isdigit(ch)) { /* it is a digit */ }
char upper = static_cast<char>(std::toupper(ch));
```

The classification functions return a value usable as Boolean; conversion functions return the converted character code or the original value when no conversion exists. Whitespace includes more than an ordinary space, such as tabs and line breaks.

---

3. C-Style Strings and `<cstring>`

3.1 Representation and the null terminator

```
char cstr[10];
char word1[] = "hello";
char word2[] = {'h', 'e', 'l', 'l', 'o', '\0'};
```

`word1` and `word2` contain the same C string. The text has five visible characters but the array requires six elements because `\0` marks the end.

For `"hello, world"`, indexes run from `0` through `11`, and `\0` occupies index `12`. String positions begin at zero.

3.2 `"a"` is not the same as `'a'`

- `'a'` is one value of type `char`.
- `"a"` is a string literal represented by an array containing `'a'` followed by `\0`; therefore it needs two `char` elements.

3.3 Initialization is not assignment

This is valid initialization:

```
char cstr[] = "world";
```

After an array has been created, these are invalid:

```
// cstr = "world";    // invalid: arrays are not assignable
// cstr[] = "world";  // invalid syntax
```

Copy into the existing array with `strcpy`, provided the destination is large enough:

```
char cstr[20];
std::strcpy(cstr, "world");
```

3.4 Example: copying and concatenation

```
#include <cstring>
#include <iostream>

int main() {
    char cstr[80];
    std::strcpy(cstr, "these ");
```

```
    std::strcat(cstr, "strings ");
    std::strcat(cstr, "are ");
    std::strcat(cstr, "concatenated.");
    std::cout << cstr << " length = " << std::strlen(cstr) << '\n';
}
```

Output:

```
these strings are concatenated. length = 31
```

Every `strcat` searches for the existing `\0`, writes the new characters there, and places a new `\0` at the end. The programmer is responsible for ensuring the array has enough capacity.

3.5 `sizeof` versus `strlen`

```
char cstr[] = "hello";
std::cout << sizeof cstr << '\n';   // 6 bytes, including \0
std::cout << std::strlen(cstr);     // 5 visible characters
```

- `sizeof cstr` gives the allocated array size when `cstr` is truly an array in the current scope.
- `strlen(cstr)` scans until the first `\0` and reports the logical string length.

The lecture's exercise then attempts `strcpy(cstr, "hello world")`. This is a buffer overflow because the six-byte array cannot hold twelve bytes (`11` visible characters plus `\0`). The behavior is **undefined**: any apparent output is unreliable, and nearby memory may be corrupted.

3.6 Broader `<cstring>` categories

The interface also includes:

- Memory operations: `memcpy`, `memmove`, `memcmp`, `memchr`
- Copying: `strcpy`, `strncpy`
- Concatenation: `strcat`, `strncat`
- Comparison and locale-related operations: `strcmp`, `strncmp`, `strcoll`, `strxfrm`
- Searching and tokenization: `strchr`, `strrchr`, `strstr`, `strpbrk`, `strspn`, `strcspn`, `strtok`

`memcpy` is for non-overlapping memory ranges; `memmove` is suitable when source and destination overlap. The bounded `strn*` functions do not automatically make code safe: correct capacity calculations and null termination are still necessary.

---

4. C++ Strings as Abstract Values

4.1 Why `std::string` is an abstraction

`std::string` lets the programmer think of a string as a value rather than as a manually managed array. Its internal representation is hidden. A class describes a type's data and behavior; an object is an instance of that class; and functions associated with the class are methods.

This follows object-oriented encapsulation: representation and operations are collected into one abstraction.

```
#include <string>

std::string name = "Ray";
```

4.2 Header meanings

Directive	Meaning
`#include <cstring>`	C string library presented for C++

Directive	Meaning
`#include <string.h>`	C-style header; accepted in C++
`#include <string>`	C++ `std::string` library
`#include "cstring.h"`	User/project header named `cstring.h`
`#include "string.h"`	Usually searches for a user/project header first
`#include <cstring.h>`	Not the standard C++ header; normally an error unless a nonstandard system header exists

Angle brackets normally select installed/system headers; quotation marks normally search the project location first.

4.3 Reading a name

```
std::string name;
std::cout << "Enter your full name: ";
std::cin >> name;
std::cout << "Hello, " << name << "!\n";
```

Despite the prompt, `std::cin >> name` stops at whitespace, so entering `Ray Chen` stores only `Ray`. To read a complete line, use:

```
std::getline(std::cin, name);
```

---

5. Operators and Common `std::string` Methods

5.1 Indexing

Both forms can read or modify a character:

```
std::string str = "Test";
str[0] = 'W';          // "West"
char c = str[0];       // 'W'
str.at(0) = 'B';       // "Best"
```

The valid indexes are `0` through `str.length() - 1` for a nonempty string.

- `str[index]` is concise but does not perform the same bounds check.
- `str.at(index)` checks the index and throws `std::out_of_range` if it is invalid.

5.2 Concatenation and operator overloading

Classes may define meanings for standard operators. For `std::string`, `+` concatenates and `+=` appends.

```
std::string first = "Hello, ";
std::string message = first + "world" + "!";
first += "AnnAnn";
```

However, `+` does not concatenate two raw C-string literals:

```
// std::string s = "hello" + ", " + "world";  // invalid
std::string s = std::string("hello") + ", " + "world";
```

At least one operand must establish a `std::string` operation at the correct point in the expression.

5.3 Why array addition behaves strangely

Given:

```
char name1[] = "Ray";
std::string name2 = "Ray";
```

These are straightforward:

```
std::cout << "Hello, " << name1 << '\n';
std::cout << "Hello, " << name2 << '\n';
std::cout << "Hello, " + name2 << '\n';
```

But `"Hello, " + name1` is invalid because both operands decay to pointers and C++ does not define pointer-plus-pointer concatenation. The `std::string` overload makes `"Hello, " + name2` valid.

5.4 Length and conversion to a C string

```
int len1 = static_cast<int>(str.length());
int len2 = static_cast<int>(std::strlen(str.c_str()));
```

The first form is the natural object-oriented solution. `c_str()` is useful when an older API specifically requires a null-terminated C string.

5.5 Searching and substrings

```
std::string s = "concatenation";
std::string part = s.substr(3, 4);  // "cate"

std::size_t p1 = s.find('a', 0);
std::size_t p2 = s.find("nation", 0);

if (p2 != std::string::npos) {
    // substring found
}
```

`string::npos` is the special value meaning no match was found. Since positions and lengths use an unsigned size type, `std::size_t` is the best type for stored indexes.

5.6 Lexicographic comparison

The comparison operators compare strings lexicographically, character by character. This resembles dictionary order but is based on character-code ordering and case matters; for example, uppercase and lowercase characters do not have identical codes.

---

6. Iterating Through Strings

6.1 Forward traversal

```
for (std::size_t i = 0; i < str.length(); ++i) {
    // use str[i]
}
```

6.2 Reverse traversal

The lecture shows an integer index:

```
std::string reverse(std::string str) {
    std::string rev;
    for (int i = static_cast<int>(str.length()) - 1; i >= 0; --i) {
        rev += str[i];
    }
```

```
    return rev;
}
```

This turns `"desserts"` into `"stressed"`. The explicit conversion prevents unsigned underflow when the string is empty.

---

7. Modifying String Contents

Unlike strings in Python, Java, and C#, C++ strings are mutable.

```
std::string s = "abcdef";
s.erase(2, 2);          // "abef"
s.insert(2, "CD");      // "abCDef"
s.replace(2, 2, "xy"); // "abxyef"
s[0] = 'A';             // "Abxyef"
```

These operations change the object itself. Mutation can be efficient, but uncontrolled changes can make a program harder to reason about.

7.1 Three case-conversion designs

A. Build a separate result: easy to understand and preserves the original, but repeated appending may perform extra allocation.

```
std::string toUpperCase(std::string str) {
    std::string result;
    for (std::size_t i = 0; i < str.length(); ++i) {
        result += static_cast<char>(std::toupper(str[i]));
    }
    return result;
}
```

B. Modify the local copy: preserves the caller's string while reusing the copied string's storage.

```
std::string toUpperCase(std::string str) {
    for (std::size_t i = 0; i < str.length(); ++i) {
        str[i] = static_cast<char>(std::toupper(str[i]));
    }
    return str;
}
```

C. Modify through a reference: avoids the input copy, but changes the caller's object and therefore has a visible side effect.

```
void toUpperCaseInPlace(std::string& str) {
    for (std::size_t i = 0; i < str.length(); ++i) {
        str[i] = static_cast<char>(std::toupper(str[i]));
    }
}
```

The tradeoff is not simply "safe versus unsafe." Passing by value gives value semantics and protects the caller's object; passing by non-const reference is efficient when mutation is intentional but requires the caller to expect that mutation.

---

8. Application: Recognizing Palindromes

A palindrome reads the same forward and backward, such as `level` or `noon`.

Direct comparison

```cpp
bool isPalindrome(const std::string& str) {
    std::size_t n = str.length();
    for (std::size_t i = 0; i < n / 2; ++i) {
        if (str[i] != str[n - i - 1]) return false;
    }
    return true;
}
```

Only half the string must be inspected. It uses constant extra space and can stop at the first mismatch.

Using `reverse`

```cpp
bool isPalindrome(std::string str) {
    return str == reverse(str);
}
```

This is shorter and very readable, but it constructs a reversed string and uses extra memory. The comparison illustrates the common tradeoff between readability and low-level efficiency.

---

9. Application: Generating Acronyms

An acronym can be formed by taking the first letter of every word:

- `self-contained underwater breathing apparatus` -> `scuba`
- `not in my back yard` -> `nimby`
- `Federal Emergency Management Agency` -> `FEMA`

A simple solution treats the first non-whitespace character as the beginning of a word:

```cpp
#include <cctype>
#include <string>

std::string acronym(const std::string& text) {
    std::string result;
    bool atWordStart = true;

    for (char ch : text) {
        if (std::isspace(static_cast<unsigned char>(ch))) {
            atWordStart = true;
        } else if (atWordStart) {
            result += ch;
            atWordStart = false;
        }
    }
    return result;
}
```

This version preserves the capitalization of the source, matching the lecture examples.

---

10. Exam and Debugging Checklist

1. Distinguish a character (`'a'`) from a string literal (`"a"`).
2. Remember that a C string requires a final `\0` .
3. Do not assign to a `char` array after initialization; copy only into sufficient storage.
4. Know that `sizeof(array)` includes its full allocated size, while `strlen` counts characters before `\0` .
5. Prefer `std::string` and its methods for normal C++ code.
6. Use `getline` rather than `operator>>` when input may contain spaces.

7. Convert at least one operand to `std::string` before using `+` on literals.
8. Check `find` results against `std::string::npos`.
9. Use `at()` when bounds checking matters; valid positions end at `length() - 1`.
10. Be careful when iterating backward because string lengths use unsigned types.
11. Understand whether a function receives a string by value, `const` reference, or mutable reference.
12. Recognize the capacity risks of `strcpy` and `strcat`; buffer overflow is undefined behavior.

---

11. Core Takeaway

C strings reveal how character arrays and null termination work, but they place memory and capacity responsibility on the programmer. `std::string` provides the more useful C++ abstraction: it supports value assignment, concatenation, comparison, searching, substring extraction, controlled mutation, and convenient integration with streams. Understanding both models is important, but `std::string` should be the default for most new C++ programs.